Fixed Bases to RBF Networks and Neural Networks, and a Bit on RKHS

1 From Basis Expansions to Neural Networks

Before introducing RKHS theory, it is helpful to understand how modern machine learning methods arise from classical basis expansions.

1.1 Univariate Basis Expansions

Suppose $x \in \mathbb{R}$ is a scalar predictor and the regression relationship is

$$y_i = f(x_i) + \varepsilon_i, \qquad i = 1, \ldots, n,$$

where f is an unknown smooth function. A standard nonparametric strategy is to approximate f by a finite basis expansion:

$$f(x) = \sum_{j=1}^{N} \beta_j B_j(x),$$

where $\{B_j(x)\}_{j=1}^{N}$ is a chosen collection of basis functions and $\beta_1, \ldots, \beta_N$ are unknown coefficients.

The basis functions determine the function class used for approximation. Common examples include:

- **Polynomial bases:**
$$B_j(x) = x^j, \qquad j = 0, 1, \ldots, N.$$
These produce global polynomial approximations. They are simple and easy to interpret, but high-degree polynomials can be numerically unstable and may behave poorly near the boundaries.

- **Fourier bases:**
$$B_{2j-1}(x) = \sin(2\pi j x), \qquad B_{2j}(x) = \cos(2\pi j x), \qquad j = 1, 2, \ldots.$$
These are well suited for periodic or approximately periodic signals. Since each basis function is global, Fourier expansions are effective for smooth oscillatory structure but less effective for highly localized features.

- **Spline bases:** Here $B_j(x)$ are spline basis functions, typically B-splines or truncated power bases, defined relative to a knot sequence. Splines are piecewise polynomials joined smoothly at knots, providing flexible local approximation while maintaining smoothness.

- **Wavelet bases:** Here $B_j(x)$ are wavelet basis functions, such as Haar or Daubechies wavelets. Wavelets are localized in both position and scale, making them especially useful for functions with local irregularities, jumps, or spatially inhomogeneous smoothness.

After fixing the basis, estimation reduces to finding the coefficients $\boldsymbol{\beta} = (\beta_1, \ldots, \beta_N)^\top$. A common approach is penalized least squares:

$$\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{N} \beta_j B_j(x_i) \right)^2 + \lambda \boldsymbol{\beta}^\top \boldsymbol{\Omega} \boldsymbol{\beta}.$$

The first term measures goodness of fit, while the penalty term controls roughness or complexity. The matrix $\boldsymbol{\Omega}$ depends on the basis and the desired notion of smoothness. For example, in spline smoothing, $\boldsymbol{\Omega}$ is often derived from an integrated squared second derivative penalty:

$$\int \{f''(x)\}^2 \, dx.$$

The tuning parameter $\lambda \geq 0$ balances fidelity to the data against smoothness. When $\lambda = 0$, the model interpolates the data closely; when λ is large, the fitted function becomes smoother and less variable.

An important point is that the model is *linear in the coefficients* $\boldsymbol{\beta}$, since

$$f(x) = \boldsymbol{\beta}^\top \boldsymbol{B}(x), \qquad \boldsymbol{B}(x) = (B_1(x), \ldots, B_N(x))^\top,$$

even though $f(x)$ can be highly nonlinear as a function of x. This linear-in-parameters representation is what makes basis expansions computationally convenient.

R Example: Polynomial, Spline, and Fourier Basis Expansions

The following code fits polynomial, natural cubic spline, and Fourier basis expansions to the same noisy data, illustrating the different function classes each basis induces.

```
set.seed(42)
n   <- 100
x   <- seq(0, 1, length = n)
f0  <- sin(2 * pi * x) + 0.5 * cos(4 * pi * x)   # true function
y   <- f0 + rnorm(n, 0, 0.3)

#   1. Polynomial basis (degree 10)
X_poly <- poly(x, degree = 10, raw = FALSE)
fit_poly <- lm(y ~ X_poly)
yhat_poly <- predict(fit_poly)

#   2. Natural cubic spline basis (10 knots)
library(splines)
X_ns  <- ns(x, df = 10)
fit_ns <- lm(y ~ X_ns)
yhat_ns <- predict(fit_ns)

#   3. Fourier basis (5 harmonics = 10 terms)
J <- 5
X_fourier <- matrix(0, n, 2 * J)
for (j in seq_len(J)) {
```

```
  X_fourier[, 2*j-1] <- sin(2 * pi * j * x)
  X_fourier[, 2*j]   <- cos(2 * pi * j * x)
}
fit_fourier <- lm(y ~ X_fourier)
yhat_fourier <- predict(fit_fourier)

#   Plot
par(mfrow = c(1, 3))
for (fit_name in c("Polynomial", "Natural spline", "Fourier")) {
  yhat <- switch(fit_name,
    Polynomial     = yhat_poly,
    "Natural spline" = yhat_ns,
    Fourier        = yhat_fourier)
  plot(x, y, pch = 16, cex = 0.5, col = "gray70",
       main = paste(fit_name, "basis"),
       xlab = "x", ylab = "y")
  lines(x, f0,   lwd = 2, col = "black", lty = 2)
  lines(x, yhat, lwd = 2, col = "steelblue")
  legend("topright", legend = c("Truth", "Fit"),
         lty = c(2, 1), col = c("black", "steelblue"), bty = "n")
}
```

1.2 Multivariate Basis Expansions

Now let $\boldsymbol{x} = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$. The same idea extends to

$$f(\boldsymbol{x}) = \sum_{j=1}^{N} \beta_j \phi_j(\boldsymbol{x}),$$

where $\phi_1, \ldots, \phi_N$ are basis functions on $\mathbb{R}^d$. A particularly common construction builds multivariate basis functions from lower-dimensional ones. Examples include:

- **Tensor-product splines:**

  $$\phi_{j_1,\ldots,j_d}(\boldsymbol{x}) = \prod_{\ell=1}^{d} B_{j_\ell}(x_\ell).$$

  These preserve the local structure of spline bases, but the number of basis functions grows as K^d when each coordinate uses K functions—an exponential blow-up in dimension.

- **Multivariate polynomial bases:** Monomials $x_1^{a_1} \cdots x_d^{a_d}$ with nonnegative integer exponents. Conceptually simple but can become large and ill-conditioned for moderate degree and dimension.

- **Multivariate Fourier bases:** Products of univariate sine and cosine terms, e.g., $\sin(2\pi j x_1)\cos(2\pi k x_2)$. Natural for periodic multivariate functions.

- **Multivariate wavelets:** Tensor products of univariate wavelets and scaling functions, providing multiscale, localized representations.

As in the univariate case, the model remains linear in the coefficients:

$$f(\boldsymbol{x}) = \boldsymbol{\beta}^\top \boldsymbol{\phi}(\boldsymbol{x}), \qquad \boldsymbol{\phi}(\boldsymbol{x}) = \big(\phi_1(\boldsymbol{x}), \ldots, \phi_N(\boldsymbol{x})\big)^\top.$$

This is called a **feature map representation**: the basis functions map $\boldsymbol{x}$ to a feature vector $\boldsymbol{\phi}(\boldsymbol{x})$, and regression is then linear in that transformed space.

The exponential growth $N \sim K^d$ is one form of the *curse of dimensionality*, and motivates structured alternatives such as additive models, low-rank tensor decompositions, sparse basis expansions, or learned feature representations.

Fixed versus adaptive bases. The spline, Fourier, and classical wavelet bases above are *fixed* once the knots, frequencies, or wavelet family are chosen; only the coefficients are estimated. In contrast, the approaches described from here onward include *learnable* parameters in the basis functions themselves, making the representation *data-adaptive*.

R Example: Tensor-Product Spline and the Curse of Dimensionality

The code below fits a bivariate tensor-product spline, then prints the number of basis terms as the dimension grows, demonstrating the exponential explosion.

```
library(splines)

#   Bivariate tensor-product spline (d = 2)
set.seed(1)
n  <- 200
x1 <- runif(n); x2 <- runif(n)
f0 <- sin(2 * pi * x1) * cos(2 * pi * x2)
y  <- f0 + rnorm(n, 0, 0.2)

B1 <- ns(x1, df = 5)          # 5 basis functions for x1
B2 <- ns(x2, df = 5)          # 5 basis functions for x2

# Tensor product: 5 x 5 = 25 basis functions
X_tp <- model.matrix(~ B1:B2 - 1)   # 25-column design matrix
fit  <- lm(y ~ X_tp)
cat("Tensor-product basis dimension:", ncol(X_tp), "\n")

#   Curse of dimensionality: basis count as d grows
K <- 5          # knots per dimension
dims <- 1:8
n_basis <- K^dims
cat("\nBasis functions K^d with K =", K, ":\n")
print(data.frame(d = dims, N_basis = n_basis))

#   Additive model alternative (mgcv)
library(mgcv)
df_2d <- data.frame(y = y, x1 = x1, x2 = x2)
fit_gam <- gam(y ~ s(x1) + s(x2), data = df_2d)
cat("\nAdditive GAM effective df:", sum(fit_gam$edf), "\n")
```

1.3 Radial Basis Function Models

Radial basis function (RBF) models use localized basis functions centered at fixed points $\boldsymbol{c}_1, \ldots, \boldsymbol{c}_M \in \mathbb{R}^d$:

$$f(\boldsymbol{x}) = \sum_{j=1}^{M} w_j \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{c}_j\|^2}{2\sigma^2}\right).$$

These models avoid the tensor-product explosion because the basis functions depend on Euclidean distance rather than coordinate-wise interactions. They are flexible approximators that extend naturally to high-dimensional inputs, and they provide the first connection to RKHS methods: the Gaussian kernel is exactly the reproducing kernel of the function space underlying Gaussian RKHS regression.

R Example: RBF Regression with Fixed Centers

```
set.seed(7)
n  <- 80
x  <- seq(0, 1, length = n)
f0 <- sin(3 * pi * x)
y  <- f0 + rnorm(n, 0, 0.2)

# Fixed centers: equally spaced
M     <- 15
c_j   <- seq(0, 1, length = M)
sigma <- 0.15     # bandwidth

# Build RBF design matrix: Phi[i,j] = exp(-||x_i - c_j||^2 / (2*sigma^2))
rbf_design <- function(x, centers, sigma) {
  outer(x, centers, FUN = function(xi, cj) {
    exp(-(xi - cj)^2 / (2 * sigma^2))
  })
}

Phi <- rbf_design(x, c_j, sigma)   # n x M matrix

# Ordinary least squares on the design matrix
fit_rbf <- lm(y ~ Phi - 1)
yhat    <- Phi %*% coef(fit_rbf)

plot(x, y, pch = 16, cex = 0.6, col = "gray70",
     main = "RBF regression (fixed centers)",
     xlab = "x", ylab = "y")
lines(x, f0,   lwd = 2, lty = 2)
lines(x, yhat, lwd = 2, col = "firebrick")
legend("topright", legend = c("Truth", "RBF fit"),
       lty = c(2, 1), col = c("black", "firebrick"), bty = "n")
```

1.4 RBF Networks

RBF models become RBF *networks* when the centers $\boldsymbol{c}_j$ are treated as free parameters estimated from data:

$$f(\boldsymbol{x}) = \sum_{j=1}^{M} w_j \, \phi(\|\boldsymbol{x} - \boldsymbol{c}_j\|).$$

Training now involves estimating $\{w_j, \boldsymbol{c}_j\}_{j=1}^{M}$, making the model *nonlinear in the parameters*. RBF networks therefore represent a transition from fixed to learned bases:

$$\text{fixed bases} \quad \longrightarrow \quad \text{learned bases.}$$

R Example: RBF Network with K-Means Centers

A practical two-stage strategy: learn the centers via k-means clustering, then fit the weights by least squares.

```
set.seed(3)
n  <- 120
x  <- runif(n, 0, 2 * pi)
y  <- sin(x) + 0.3 * rnorm(n)

# Stage 1: learn M centers by k-means
M       <- 12
km      <- kmeans(x, centers = M, nstart = 20)
centers <- sort(km$centers)   # M learned centers
sigma   <- 0.5

# Stage 2: build design matrix and fit weights by OLS
Phi  <- rbf_design(x, centers, sigma)   # reuse function from above
w    <- coef(lm(y ~ Phi - 1))

# Prediction on a fine grid
x_grid <- seq(0, 2 * pi, length = 300)
Phi_g  <- rbf_design(x_grid, centers, sigma)
yhat_g <- Phi_g %*% w

plot(x, y, pch = 16, cex = 0.6, col = "gray70",
     main = "RBF network (k-means centers)",
     xlab = "x", ylab = "y")
lines(x_grid, sin(x_grid), lwd = 2, lty = 2)
lines(x_grid, yhat_g,      lwd = 2, col = "darkgreen")
abline(v = centers, col = "darkgreen", lty = 3, lwd = 0.8)
legend("topright", legend = c("Truth", "RBF network", "Centers"),
       lty = c(2, 1, 3), col = c("black", "darkgreen", "darkgreen"),
       bty = "n")
```

1.5 Connection with Gaussian Mixture Models

When the radial basis function is Gaussian,

$$f(\boldsymbol{x}) = \sum_{j=1}^{M} w_j \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{c}_j\|^2}{2\sigma^2}\right),$$

the structure closely resembles a Gaussian mixture model:

$$p(\boldsymbol{x}) = \sum_{j=1}^{M} \pi_j \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j),$$

where each component $\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j) \propto \exp\!\big(-\frac{1}{2}(\boldsymbol{x} - \boldsymbol{\mu}_j)^\top \boldsymbol{\Sigma}_j^{-1}(\boldsymbol{x} - \boldsymbol{\mu}_j)\big)$ has the same functional form as a Gaussian radial basis function. The key difference lies in the objective:

	Gaussian Mixture Model	RBF Network
Goal	Density estimation	Regression
Output	$p(\boldsymbol{x})$	$f(\boldsymbol{x})$
Weights	$\pi_j \geq 0$, sum to 1	Arbitrary w_j
Training	EM algorithm	Least squares / SGD

Gaussian mixtures and RBF networks use identical Gaussian building blocks, but the former models densities while the latter approximates regression functions—a transition from unsupervised density modeling to supervised function approximation.

R Example: Gaussian Mixture vs RBF Network on the Same Data

```
library(mclust)   # EM-based Gaussian mixture models

set.seed(99)
n  <- 200
x  <- c(rnorm(100, -2, 0.8), rnorm(100, 2, 0.8))
y  <- sin(x) + 0.3 * rnorm(n)

#   GMM: unsupervised density estimation
gmm <- densityMclust(x, G = 3, plot = FALSE)

#   RBF network: supervised regression
# Reuse GMM component means as RBF centers
centers_gmm <- gmm$parameters$mean
sigma_rbf   <- 0.8

Phi_gmm  <- rbf_design(x, centers_gmm, sigma_rbf)
w_rbf    <- coef(lm(y ~ Phi_gmm - 1))

x_grid  <- seq(-5, 5, length = 300)
Phi_g   <- rbf_design(x_grid, centers_gmm, sigma_rbf)
yhat_g  <- Phi_g %*% w_rbf
```

```
#   Plot both
par(mfrow = c(1, 2))

# GMM density
plot(gmm, what = "density", main = "GMM density estimate",
     xlab = "x")
rug(x, col = "gray50")

# RBF regression
plot(x, y, pch = 16, cex = 0.6, col = "gray70",
     main = "RBF network regression",
     xlab = "x", ylab = "y")
lines(x_grid, sin(x_grid), lwd = 2, lty = 2)
lines(x_grid, yhat_g,      lwd = 2, col = "purple")
abline(v = centers_gmm, col = "purple", lty = 3)
legend("topright", legend = c("Truth", "RBF fit"),
       lty = c(2, 1), col = c("black", "purple"), bty = "n")
```

1.6 Feedforward Neural Networks

General feedforward neural networks replace radial functions with nonlinear activations applied to linear projections:

$$f(\boldsymbol{x}) = \sum_{j=1}^{M} a_j \, \sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j).$$

Here $\boldsymbol{w}_j^\top \boldsymbol{x} + b_j$ defines a learned linear feature, $\sigma(\cdot)$ is a nonlinear activation (for certain choices of σ, $\boldsymbol{w}_j$, and b_j one recovers radial basis functions), and a_j are output weights. Neural networks are therefore **adaptive basis expansions** with basis functions

$$\phi_j(\boldsymbol{x}) = \sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)$$

learned from the data rather than fixed in advance.

1.7 Deep Neural Networks

Deep neural networks extend this idea by composing multiple layers of learned features:

$$f(\boldsymbol{x}) = \boldsymbol{W}_L \, \sigma\big(\boldsymbol{W}_{L-1} \, \sigma(\cdots \sigma(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) \cdots) + \boldsymbol{b}_{L-1}\big) + \boldsymbol{b}_L.$$

Each layer produces a new feature representation, leading to hierarchical function approximation. This gives the progression:

$$\text{fixed basis expansion} \;\rightarrow\; \text{RBF models} \;\rightarrow\; \text{shallow neural networks} \;\rightarrow\; \text{deep neural networks}.$$

> Neural networks are best understood as adaptive multivariate basis expansions. RKHS theory provides the mathematical framework connecting these basis expansions to kernel methods.

R Example: Shallow and Deep Networks via `nnet` and `keras`

```
#   Shallow network (1 hidden layer) with nnet
library(nnet)
set.seed(5)
n  <- 150
x  <- runif(n, 0, 2 * pi)
y  <- sin(x) + 0.2 * rnorm(n)

fit_shallow <- nnet(x, y, size = 10, linout = TRUE,
                    maxit = 500, decay = 1e-3, trace = FALSE)
yhat_shallow <- predict(fit_shallow, data.frame(x = x))

#   Deep network (3 hidden layers) with keras / tensorflow
# Requires:  install.packages("keras"); keras::install_keras()
library(keras)

x_sc <- scale(x)    # standardise input
y_sc <- scale(y)

model <- keras_model_sequential() |>
  layer_dense(units = 32, activation = "relu",
              input_shape = 1) |>
  layer_dense(units = 32, activation = "relu") |>
  layer_dense(units = 16, activation = "relu") |>
  layer_dense(units = 1)

model |> compile(optimizer = "adam", loss = "mse")

model |> fit(
  x = matrix(x_sc), y = matrix(y_sc),
  epochs = 300, batch_size = 32, verbose = 0
)

yhat_deep <- predict(model, matrix(x_sc)) *
             attr(y_sc, "scaled:scale") +
             attr(y_sc, "scaled:center")

#   Compare
par(mfrow = c(1, 2))
ord <- order(x)

plot(x[ord], y[ord], pch = 16, cex = 0.5, col = "gray70",
     main = "Shallow network (nnet, 1 hidden layer)",
     xlab = "x", ylab = "y")
lines(x[ord], sin(x[ord]),          lwd = 2, lty = 2)
lines(x[ord], yhat_shallow[ord],    lwd = 2, col = "darkorange")
```

```
plot(x[ord], y[ord], pch = 16, cex = 0.5, col = "gray70",
     main = "Deep network (keras, 3 hidden layers)",
     xlab = "x", ylab = "y")
lines(x[ord], sin(x[ord]),       lwd = 2, lty = 2)
lines(x[ord], yhat_deep[ord],    lwd = 2, col = "steelblue")
```

2 RKHS Motivation

Modern machine learning methods—kernel machines, RBF networks, and neural networks—are often presented as fundamentally different. In reality, they share a common foundation in *function approximation*.

In many learning problems we observe data $(x_1, y_1), \dots, (x_n, y_n)$ and wish to estimate $f : \mathcal{X} \to \mathbb{R}$. Unlike classical parametric regression, which estimates a finite vector $\boldsymbol{\beta} \in \mathbb{R}^p$, modern methods estimate an entire function, so the parameter space is typically *infinite-dimensional*. Many learning algorithms can be written as

$$\min_{f \in \mathcal{F}} \left\{ \sum_{i=1}^{n} L(y_i, f(x_i)) + \lambda J(f) \right\},$$

where L measures goodness of fit, $J(f)$ measures complexity or roughness, and λ controls regularization. Reproducing Kernel Hilbert Space (RKHS) theory provides the mathematical framework that unifies kernel methods, RBF networks, and many neural network constructions, explaining why kernels work, why regularization is necessary, why solutions often have simple finite representations, and how classical smoothing methods relate to modern machine learning.

2.1 Infinite-Width Neural Networks and Gaussian Process Limits

Consider a single-hidden-layer network of width m:

$$f_m(x) = \frac{1}{\sqrt{m}} \sum_{j=1}^{m} a_j \, \sigma(\boldsymbol{w}_j^\top x + b_j),$$

with parameters drawn independently as

$$\boldsymbol{w}_j \sim \mathcal{N}(0, \boldsymbol{\Sigma}_w), \qquad b_j \sim \mathcal{N}(0, \sigma_b^2), \qquad a_j \sim \mathcal{N}(0, \sigma_a^2),$$

and a measurable activation $\sigma : \mathbb{R} \to \mathbb{R}$ satisfying $\mathbb{E}[\sigma(Z)^2] < \infty$ for $Z \sim \mathcal{N}(0, 1)$.

Theorem 2.1 (Infinite-width neural network limit)**.** *Let $x_1, \dots, x_n \in \mathbb{R}^d$ be fixed inputs. As $m \to \infty$,*

$$\big(f_m(x_1), \dots, f_m(x_n)\big) \xrightarrow{d} \mathcal{N}(0, K),$$

where the (i, j) entry of K is

$$K(x, x') = \sigma_a^2 \, \mathbb{E}_{\boldsymbol{w}, b}\big[\sigma(\boldsymbol{w}^\top x + b) \, \sigma(\boldsymbol{w}^\top x' + b)\big].$$

Consequently, $f_m(\cdot)$ converges in finite-dimensional distributions to a Gaussian process $f(\cdot) \sim \mathcal{GP}(0, K)$.

Interpretation. The infinite-width network induces a GP prior with covariance kernel

$$K(x, x') = \mathbb{E}[\phi(x)\phi(x')], \qquad \phi(x) = \sigma(\boldsymbol{w}^\top x + b),$$

and the corresponding function space is the RKHS generated by K.

Example 1: Linear activation. $\sigma(z) = z$ gives $K(x, x') = x^\top x'$, reducing to a linear kernel model.

Example 2: ReLU activation. For $\sigma(z) = \max(0, z)$ and $\boldsymbol{w} \sim \mathcal{N}(0, I)$,

$$K(x, x') = \frac{\|x\|\|x'\|}{2\pi}\Big(\sin\theta + (\pi - \theta)\cos\theta\Big), \qquad \theta = \cos^{-1}\left(\frac{x^\top x'}{\|x\|\|x'\|}\right).$$

This is the *arc-cosine kernel* [Cho and Saul, 2009].

Example 3: Tanh activation. $\sigma(z) = \tanh(z)$ gives

$$K(x, x') = \mathbb{E}[\tanh(u)\tanh(v)], \qquad (u, v) \sim \mathcal{N}\left(0, \begin{pmatrix} x^\top x & x^\top x' \\ x^\top x' & x'^\top x' \end{pmatrix}\right).$$

Deep networks. For a depth-L network, the covariance satisfies the layer-wise recursion

$$K^{(\ell+1)}(x, x') = \sigma_a^2\, \mathbb{E}_{(u,v)}[\sigma(u)\sigma(v)] + \sigma_b^2, \quad (u, v) \sim \mathcal{N}\left(0, \begin{pmatrix} K^{(\ell)}(x, x) & K^{(\ell)}(x, x') \\ K^{(\ell)}(x, x') & K^{(\ell)}(x', x') \end{pmatrix}\right),$$

initialized at the input kernel $K^{(0)}(x, x') = x^\top x'$.

References. Early results connecting infinite-width networks to Gaussian processes appear in Neal [1996] and Williams [1997]. Explicit neural network kernels were studied in Cho and Saul [2009], and the deep network GP limit was established in Lee et al. [2018]. There is a growing literature showing that ReLU-based deep networks admit a spline-based characterization; see the works of Baraniuk, Schmidt-Hieber, and colleagues for theoretical developments along these lines.

R Example: Simulating the Infinite-Width GP Limit

The code below verifies Theorem 2.1 empirically: as the width m grows, the empirical covariance of the network output converges to the theoretical NNGP kernel.

```
set.seed(21)
relu  <- function(z) pmax(0, z)
sigma_a <- 1; sigma_b <- 0.5; sigma_w <- 1

# Theoretical arc-cosine kernel for ReLU
nngp_relu_kernel <- function(x1, x2, sigma_a = 1, sigma_w = 1) {
  K11 <- sigma_w^2 * sum(x1^2) + sigma_b^2
  K22 <- sigma_w^2 * sum(x2^2) + sigma_b^2
  K12 <- sigma_w^2 * sum(x1 * x2) + sigma_b^2
  theta <- acos(pmin(pmax(K12 / sqrt(K11 * K22), -1), 1))
```

```
  sigma_a^2 / (2 * pi) * sqrt(K11 * K22) *
    (sin(theta) + (pi - theta) * cos(theta))
}

x1 <- c(1, 0); x2 <- c(0.5, 0.5)

# Empirical covariance at widths m = 10, 100, 1000, 10000
widths <- c(10, 100, 1000, 10000)
K_emp  <- numeric(length(widths))
B      <- 5000   # Monte Carlo samples

for (idx in seq_along(widths)) {
  m   <- widths[idx]
  out <- replicate(B, {
    W  <- matrix(rnorm(m * 2, 0, sigma_w), m, 2)
    b  <- rnorm(m, 0, sigma_b)
    a  <- rnorm(m, 0, sigma_a)
    f1 <- sum(a * relu(W %*% x1 + b)) / sqrt(m)
    f2 <- sum(a * relu(W %*% x2 + b)) / sqrt(m)
    c(f1, f2)
  })
  K_emp[idx] <- cov(out[1, ], out[2, ])
}

K_theory <- nngp_relu_kernel(x1, x2)
cat("Theoretical NNGP kernel K(x1, x2):", round(K_theory, 4), "\n\n")
print(data.frame(Width = widths,
                 Empirical_K = round(K_emp, 4),
                 Error = round(abs(K_emp - K_theory), 4)))
```

3 Gaussian Processes and Their Connection to RKHS

Gaussian processes provide a flexible nonparametric framework for modeling unknown functions, and are closely connected to kernel methods and RKHS.

3.1 Gaussian Processes

Definition 3.1. A stochastic process $\{f(x) : x \in \mathcal{X}\}$ is called a *Gaussian process* if for every finite collection $x_1, \ldots, x_n \in \mathcal{X}$, the vector $(f(x_1), \ldots, f(x_n))$ has a multivariate normal distribution.

A Gaussian process is completely determined by its mean function $m(x) = \mathbb{E}[f(x)]$ and covariance function $K(x, x') = \mathrm{Cov}(f(x), f(x'))$. We write $f \sim \mathcal{GP}(m, K)$, and in most applications we take $m \equiv 0$.

3.2 Positive Definite Kernels and RKHS

The covariance function K must be *positive definite*: for all $x_1, \ldots, x_n$ and $c_1, \ldots, c_n \in \mathbb{R}$,

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j K(x_i, x_j) \geq 0.$$

Theorem 3.1 (Moore–Aronszajn)**.** *For every positive definite kernel K on $\mathcal{X}$ there exists a unique Hilbert space $\mathcal{H}_K$ of functions on $\mathcal{X}$ such that*

$$f(x) = \langle f, K(\cdot, x) \rangle_{\mathcal{H}_K} \qquad \text{for all } f \in \mathcal{H}_K.$$

The space $\mathcal{H}_K$ is the RKHS associated with K. Thus the covariance function of a GP simultaneously defines a stochastic process prior and a deterministic function space $\mathcal{H}_K$.

3.3 Sampling from a Gaussian Process

For inputs $x_1, \ldots, x_n$ define $K_{ij} = K(x_i, x_j)$. Then

$$(f(x_1), \ldots, f(x_n))^\top \sim \mathcal{N}(0, K).$$

Sampling a GP therefore reduces to sampling from a multivariate normal whose covariance is determined by the kernel, which controls the smoothness and structure of the sampled functions.

R Example: Visualising How the Kernel Controls Smoothness

```
library(MASS)

# Squared exponential kernel with length-scale l
se_kernel <- function(x1, x2, l = 1, sf = 1)
  sf^2 * exp(-outer(x1, x2, "-")^2 / (2 * l^2))

x <- seq(0, 5, length = 200)

par(mfrow = c(1, 3))
for (l in c(0.2, 0.7, 2.0)) {
  K <- se_kernel(x, x, l = l)
  K <- K + 1e-8 * diag(nrow(K))   # jitter for numerical stability
  set.seed(1)
  f <- mvrnorm(3, mu = rep(0, 200), Sigma = K)
  matplot(x, t(f), type = "l", lty = 1, lwd = 1.5,
          main = paste0("SE kernel, l = ", l),
          xlab = "x", ylab = "f(x)",
          ylim = c(-3, 3))
}
```

3.4 Example: Squared Exponential Kernel

The squared exponential (SE) kernel

$$K(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2} \right)$$

induces an RKHS of infinitely differentiable functions. The length-scale ℓ controls the rate of variation: larger ℓ yields smoother functions.

3.5 Gaussian Process Regression

Suppose $y_i = f(x_i) + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$. Let $\boldsymbol{y} = (y_1, \ldots, y_n)^\top$ and define

$$K = K(X, X), \qquad \boldsymbol{k}_* = K(X, x_*), \qquad K_{**} = K(x_*, x_*).$$

The posterior predictive distribution at a test point x_* is $f(x_*) \mid \boldsymbol{y} \sim \mathcal{N}(\mu_*, \sigma_*^2)$, where

$$\mu_* = \boldsymbol{k}_*^\top (K + \sigma^2 I)^{-1} \boldsymbol{y}, \qquad \sigma_*^2 = K_{**} - \boldsymbol{k}_*^\top (K + \sigma^2 I)^{-1} \boldsymbol{k}_*.$$

3.6 Connection with Kernel Ridge Regression

The posterior mean $\hat{f}(x) = K(x, X)^\top (K + \sigma^2 I)^{-1} \boldsymbol{y}$ coincides with the minimizer of

$$\min_{f \in \mathcal{H}_K} \sum_{i=1}^{n} (y_i - f(x_i))^2 + \lambda \|f\|_{\mathcal{H}_K}^2$$

with $\lambda = \sigma^2$. Thus GP regression is the Bayesian counterpart of kernel ridge regression, with the RKHS norm acting as a smoothness penalty. This duality shows that

$$\text{Gaussian process priors} \quad \longleftrightarrow \quad \text{RKHS regularization.}$$

From a Bayesian perspective one places a prior on functions via the GP; from a frequentist perspective one estimates functions in an RKHS via penalized least squares. The resulting estimator is the same.

R Example: GP Regression from Scratch and via `kernlab`

```
library(MASS)
library(kernlab)

set.seed(1)
se_kernel <- function(x1, x2, l = 0.3, sf = 1)
  sf^2 * exp(-outer(x1, x2, "-")^2 / (2 * l^2))

# Training data
x_train     <- seq(0, 1, length = 10)
y_train     <- sin(2 * pi * x_train) + rnorm(10, 0, 0.1)
x_test      <- seq(0, 1, length = 200)
sigma_noise <- 0.1

#   Manual GP posterior
K    <- se_kernel(x_train, x_train)
K_s  <- se_kernel(x_train, x_test)
K_ss <- se_kernel(x_test,  x_test)
L    <- K + sigma_noise^2 * diag(length(x_train))
```

```
mu   <- t(K_s) %*% solve(L) %*% y_train
vv   <- diag(K_ss - t(K_s) %*% solve(L) %*% K_s)
sd   <- sqrt(pmax(vv, 0))

#   kernlab GP
rbf_k <- rbfdot(sigma = 1 / (2 * 0.3^2))  # matches l = 0.3
gp_fit <- gausspr(x_train, y_train,
                  kernel = rbf_k, var = sigma_noise^2)
mu_kl <- predict(gp_fit, x_test)

#   Plot
par(mfrow = c(1, 2))

plot(x_test, mu, type = "l", lwd = 2,
     ylim = c(-1.8, 1.8),
     main = "GP regression (manual)",
     xlab = "x", ylab = "f(x)")
polygon(c(x_test, rev(x_test)),
        c(mu + 2*sd, rev(mu - 2*sd)),
        col = adjustcolor("steelblue", 0.2), border = NA)
lines(x_test, mu + 2*sd, lty = 2, col = "steelblue")
lines(x_test, mu - 2*sd, lty = 2, col = "steelblue")
points(x_train, y_train, pch = 19)

plot(x_test, mu_kl, type = "l", lwd = 2,
     ylim = c(-1.8, 1.8),
     main = "GP regression (kernlab)",
     xlab = "x", ylab = "f(x)")
lines(x_test, sin(2 * pi * x_test), lwd = 1.5, lty = 2)
points(x_train, y_train, pch = 19)
legend("topright", legend = c("Truth", "kernlab GP"),
       lty = c(2, 1), bty = "n")
```

4 Hilbert Spaces and Function Spaces

Machine learning algorithms often optimize over spaces of functions. To do this systematically, we need notions of distance, angles, and convergence between functions. Hilbert spaces provide exactly this structure.

Definition 4.1 (Hilbert Space)**.** A Hilbert space $(\mathcal{H}, \langle \cdot, \cdot \rangle)$ is a vector space equipped with an inner product such that every Cauchy sequence converges within the space.

The inner product defines a norm $\|f\|_{\mathcal{H}} = \sqrt{\langle f, f \rangle}$, allowing us to measure the size of and distance between functions.

Examples of Inner Products in Hilbert Spaces

1. Euclidean space $\mathbb{R}^p$. $\langle \boldsymbol{x}, \boldsymbol{y} \rangle = \sum_{j=1}^{p} x_j y_j$. The simplest Hilbert space.

2. Square-integrable functions $L^2(\Omega)$. $\langle f, g\rangle = \int_\Omega f(x)g(x)\,dx$. A fundamental infinite-dimensional Hilbert space.

3. Sequence space ℓ^2. $\langle x, y\rangle = \sum_{j=1}^{\infty} x_j y_j$ for square-summable sequences. The natural infinite-dimensional analogue of $\mathbb{R}^p$.

4. Sobolev space H^1. $\langle f, g\rangle = \int f(x)g(x)\,dx + \int f'(x)g'(x)\,dx$. This inner product measures both function size and smoothness, and plays an important role in smoothing splines and penalized regression.

5. Hölder spaces (not Hilbert spaces). For $\alpha \in (0, 1]$, the Hölder space $C^\alpha(\Omega)$ consists of functions satisfying

$$\|f\|_{C^\alpha} = \sup_x |f(x)| + \sup_{x \neq y} \frac{|f(x) - f(y)|}{|x - y|^\alpha} < \infty.$$

Although $C^\alpha(\Omega)$ is a complete normed space (a Banach space), it generally does *not* admit a natural inner product, so it is not a Hilbert space. Hölder spaces measure local smoothness, while Hilbert spaces measure smoothness through quadratic averages; RKHS theory relies on Hilbert-space structure, which is why Sobolev and RKHS spaces are preferred in learning theory.

Completeness guarantees that limits of optimization sequences remain inside the space, which is essential for the existence of minimizers.

R Example: Computing Inner Products in $\mathbb{R}^p$, L^2, and H^1

```
#   1. Euclidean inner product in R^p
u <- c(1, 2, 3, 4, 5)
v <- c(5, 4, 3, 2, 1)
ip_euclidean <- sum(u * v)
cat("Euclidean <u,v> =", ip_euclidean, "\n")   # = 35

#   2. L^2 inner product via numerical integration
x  <- seq(0, 1, length = 1000)
dx <- x[2] - x[1]
f  <- sin(pi * x)
g  <- cos(pi * x)
ip_L2 <- sum(f * g) * dx
cat("L^2 <sin, cos> =", round(ip_L2, 6), "\n")   #   0

# L^2 norms
norm_f <- sqrt(sum(f^2) * dx)
norm_g <- sqrt(sum(g^2) * dx)
cat("||sin||_L2 =", round(norm_f, 4),
    "  ||cos||_L2 =", round(norm_g, 4), "\n")   # both   1/sqrt(2)

#   3. H^1 Sobolev inner product
# Approximate derivatives with finite differences
fp <- diff(f) / dx; gp <- diff(g) / dx
xm <- x[-1]   # midpoints
ip_H1 <- sum(f[-1] * g[-1]) * dx + sum(fp * gp) * dx
cat("H^1 <sin, cos> =", round(ip_H1, 6), "\n")
```

```
#   4. Visualise the reproducing property of a linear kernel
# K(x, x') = 1 + x*x'; RKHS = {a + b*x}; <f,g>_H = a_f*a_g + b_f*b_g
f_fun  <- function(t, a =  2, b =  3) a + b * t   # f(x) = 2 + 3x
K_fun  <- function(t, x0 = 1.5)       1 + x0 * t   # K(., x0)

# Inner product <f, K(.,x0)> should equal f(x0) = 2 + 3*1.5 = 6.5
a_f <- 2; b_f <- 3; a_K <- 1; b_K <- 1.5
ip_RKHS <- a_f * a_K + b_f * b_K
cat("\nRKHS inner product <f, K(.,1.5)> =", ip_RKHS, "\n")
cat("f(1.5) =", f_fun(1.5), "\n")   # reproducing property: both = 6.5
```

5 Reproducing Kernel Hilbert Spaces (RKHS)

General Hilbert spaces of functions are very broad. In machine learning we need the additional ability to evaluate functions pointwise.

Definition 5.1 (Reproducing Kernel Hilbert Space)**.** Let $\mathcal{X}$ be a set and $\mathcal{H}$ a Hilbert space of functions $f : \mathcal{X} \to \mathbb{R}$. Then $\mathcal{H}$ is a *reproducing kernel Hilbert space (RKHS)* if for every $x \in \mathcal{X}$ the point evaluation functional $L_x(f) = f(x)$ is a continuous linear functional on $\mathcal{H}$.

Equivalently, $\mathcal{H}$ is an RKHS if for each $x \in \mathcal{X}$ there exists $K(\cdot, x) \in \mathcal{H}$ such that

$$f(x) = \langle f, K(\cdot, x)\rangle_{\mathcal{H}} \qquad \text{for all } f \in \mathcal{H}.$$

The symmetric function $K(x, x') = \langle K(\cdot, x), K(\cdot, x')\rangle_{\mathcal{H}}$ is the *reproducing kernel* of $\mathcal{H}$.

Reproducing property. The identity $f(x) = \langle f, K(\cdot, x)\rangle_{\mathcal{H}}$ means evaluating f at x is an inner product with $K(\cdot, x)$. The kernel function $K(\cdot, x)$ acts as a "basis function centered at x" that probes the value of f.

Example. Consider $K(x, x') = 1 + xx'$ on $\mathcal{X} = \mathbb{R}$. The associated RKHS consists of affine functions $f(x) = a + bx$ with inner product $\langle f, g\rangle_{\mathcal{H}} = a_f a_g + b_f b_g$. Verification of the reproducing property:

$$\langle a + bt,\ 1 + xt\rangle_{\mathcal{H}} = a \cdot 1 + b \cdot x = f(x).$$

And the kernel reproduces itself:

$$\langle K(\cdot, x), K(\cdot, x')\rangle_{\mathcal{H}} = \langle 1 + xt,\ 1 + x't\rangle_{\mathcal{H}} = 1 + xx' = K(x, x').$$

Interpretation

- $f(x) = \langle f, K(\cdot, x)\rangle_{\mathcal{H}}$: pointwise evaluation is an inner product.
- $K(\cdot, x)$ is a basis function centered at x.
- Functions in the RKHS can be built as linear combinations $f(\cdot) = \sum_{i=1}^{n} \alpha_i K(\cdot, x_i)$.
- Learning in RKHS therefore reduces to estimating the coefficients $\{\alpha_i\}$.

5.1 Positive Definite Kernels

A function $K : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ defines an RKHS if and only if it is **positive definite**:

$$\sum_{i,j=1}^{n} \alpha_i \alpha_j K(x_i, x_j) \geq 0 \qquad \text{for all } \alpha_1, \ldots, \alpha_n \in \mathbb{R}.$$

This ensures the kernel matrix $K_{ij} = K(x_i, x_j)$ is always positive semi-definite, guaranteeing stable optimization.

Common Kernels

- **Linear:** $K(\boldsymbol{x}, \boldsymbol{x}') = \boldsymbol{x}^\top \boldsymbol{x}'$. Reproduces linear regression.
- **Polynomial:** $K(\boldsymbol{x}, \boldsymbol{x}') = (\boldsymbol{x}^\top \boldsymbol{x}' + c)^d$. Corresponds to polynomial regression in a high-dimensional feature space.
- **Gaussian (RBF):** $K(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\|\boldsymbol{x} - \boldsymbol{x}'\|^2 / 2\sigma^2)$. Induces infinitely smooth functions; widely used.

Choosing a kernel is equivalent to choosing the function space in which learning takes place.

R Example: Verifying Positive Definiteness and Comparing Kernel Matrices

```
x <- seq(0, 1, length = 8)

# Three kernels
k_linear <- function(x1, x2) outer(x1, x2, "*")
k_poly   <- function(x1, x2, d = 3, c = 1)
  (outer(x1, x2, "*") + c)^d
k_rbf    <- function(x1, x2, sigma = 0.3)
  exp(-outer(x1, x2, function(a, b) (a-b)^2) / (2*sigma^2))

K_lin  <- k_linear(x, x)
K_poly <- k_poly(x, x)
K_rbf  <- k_rbf(x, x)

# Check positive semi-definiteness via eigenvalues
check_pd <- function(K, name) {
  ev <- eigen(K, symmetric = TRUE)$values
  cat(name, ": min eigenvalue =", round(min(ev), 6),
      " -> PSD:", all(ev >= -1e-10), "\n")
}
check_pd(K_lin,  "Linear kernel ")
check_pd(K_poly, "Poly kernel   ")
check_pd(K_rbf,  "RBF kernel    ")

# Visualise the kernel matrices
par(mfrow = c(1, 3))
for (nm in c("Linear", "Polynomial", "RBF")) {
```

```
  K <- switch(nm, Linear = K_lin, Polynomial = K_poly, RBF = K_rbf)
  image(K, main = paste(nm, "kernel matrix"),
        col = hcl.colors(64, "YlOrRd", rev = TRUE))
}
```

6 Learning in RKHS and the Representer Theorem

Consider the regularized risk minimization problem:

$$\min_{f \in \mathcal{H}} \left\{ \sum_{i=1}^{n} L(y_i, f(\boldsymbol{x}_i)) + \lambda \|f\|_{\mathcal{H}}^2 \right\}.$$

Theorem 6.1 (Representer Theorem)**.** *Any minimizer $f^\star \in \mathcal{H}$ admits the finite expansion*

$$f^\star(\boldsymbol{x}) = \sum_{i=1}^{n} \alpha_i \, K(\boldsymbol{x}_i, \boldsymbol{x}).$$

Remark 6.1. This result is fundamental. Optimization over the infinite-dimensional space $\mathcal{H}$ reduces to estimating n real coefficients $\alpha_1, \ldots, \alpha_n$. It explains why kernel methods scale with the number of training points n, and why the representer is supported only on the observed data.

R Example: Kernel Ridge Regression and the Representer Coefficients

Kernel ridge regression (KRR) directly implements the Representer Theorem: the solution is $f^\star(\boldsymbol{x}) = \sum_i \alpha_i K(\boldsymbol{x}_i, \boldsymbol{x})$ with $= (K + \lambda I)^{-1} \boldsymbol{y}$.

```
set.seed(8)
n       <- 60
x_train <- runif(n, 0, 2 * pi)
y_train <- sin(x_train) + 0.2 * rnorm(n)
x_test  <- seq(0, 2 * pi, length = 300)

# RBF kernel
k_rbf <- function(x1, x2, sigma = 0.5)
  exp(-outer(x1, x2, function(a,b) (a-b)^2) / (2*sigma^2))

K      <- k_rbf(x_train, x_train)
lambda <- 0.01

# Representer coefficients: alpha = (K + lambda*I)^{-1} y
alpha  <- solve(K + lambda * diag(n)) %*% y_train

# Prediction: f*(x_*) = K(x_*, X) alpha
K_star <- k_rbf(x_test, x_train)
yhat   <- K_star %*% alpha

# Compare with kernlab ksvm / gausspr
library(kernlab)
```

```
rbf_k  <- rbfdot(sigma = 1 / (2 * 0.5^2))
gp_fit <- gausspr(x_train, y_train,
                  kernel = rbf_k, var = lambda)
yhat_kl <- predict(gp_fit, x_test)

# Plot
plot(x_train, y_train, pch = 16, cex = 0.6, col = "gray60",
     main = "Kernel ridge regression (Representer Theorem)",
     xlab = "x", ylab = "y")
lines(x_test, sin(x_test),  lwd = 2, lty = 2)
lines(x_test, yhat,         lwd = 2, col = "firebrick")
lines(x_test, yhat_kl,      lwd = 2, col = "steelblue", lty = 3)
legend("topright",
       legend = c("Truth", "KRR (manual alpha)", "kernlab GP"),
       lty = c(2, 1, 3),
       col = c("black", "firebrick", "steelblue"), bty = "n")

# Show the representer coefficients
cat("\nFirst 6 representer coefficients alpha:\n")
print(round(head(alpha), 4))
```

7 Gaussian RKHS and Smoothness

The Gaussian (RBF) kernel

$$K(\boldsymbol{x}, \boldsymbol{x}') = \exp\left(-\frac{\|\boldsymbol{x} - \boldsymbol{x}'\|^2}{2\sigma^2}\right)$$

induces an RKHS of analytic functions with the following properties:

- The RKHS is infinite-dimensional.
- It is a universal approximator on compact sets.
- The RKHS norm $\|f\|_{\mathcal{H}}$ provides strong smoothness control, with the bandwidth σ governing the length scale of allowed variation.

R Example: Effect of Bandwidth on RKHS Smoothness and KRR Fit

```
set.seed(11)
n       <- 80
x_train <- runif(n, 0, 1)
y_train <- sin(4 * pi * x_train) + 0.3 * rnorm(n)
x_test  <- seq(0, 1, length = 300)

k_rbf  <- function(x1, x2, sigma)
  exp(-outer(x1, x2, function(a,b)(a-b)^2) / (2*sigma^2))

lambda <- 0.01
sigmas <- c(0.05, 0.2, 0.8)   # narrow → wide bandwidth
```

```
par(mfrow = c(1, 3))
for (sg in sigmas) {
  K      <- k_rbf(x_train, x_train, sg)
  alpha  <- solve(K + lambda * diag(n)) %*% y_train
  K_star <- k_rbf(x_test, x_train, sg)
  yhat   <- K_star %*% alpha

  # RKHS norm: ||f||^2_H = alpha' K alpha
  rkhs_norm <- sqrt(as.numeric(t(alpha) %*% K %*% alpha))

  plot(x_train, y_train, pch = 16, cex = 0.5, col = "gray70",
       main = bquote(sigma == .(sg) ~
                     "  ||f||"[H] == .(round(rkhs_norm, 2))),
       xlab = "x", ylab = "y", ylim = c(-2, 2))
  lines(x_test, sin(4 * pi * x_test), lwd = 2, lty = 2)
  lines(x_test, yhat, lwd = 2, col = "darkorange")
}
```

8 RBF Networks as Kernel Expansions

A Gaussian RBF network with centers $\boldsymbol{c}_j = \boldsymbol{x}_j$ set equal to the training points takes the form

$$f(\boldsymbol{x}) = \sum_{j=1}^{n} \alpha_j \, K(\boldsymbol{x}_j, \boldsymbol{x}),$$

which is exactly the expansion guaranteed by the Representer Theorem.

Remark 8.1. RBF networks with centers at the training points are finite-dimensional realizations of Gaussian RKHS regression.

The differences between RKHS kernel methods and general RBF networks are:

	RKHS Kernel Method	RBF Network
Centers	All n training points	Learned or fixed subset of size $M \ll n$
Parameters	n coefficients	M weights + M centers
Training	Convex	Often nonconvex

R Example: Comparing Full KRR with a Sparse RBF Approximation

```
set.seed(17)
n       <- 150
x_train <- runif(n, 0, 2 * pi)
y_train <- sin(x_train) + 0.25 * rnorm(n)
x_test  <- seq(0, 2 * pi, length = 400)

sg     <- 0.4
lambda <- 0.005
k_rbf  <- function(x1, x2)
```

```
  exp(-outer(x1, x2, function(a,b)(a-b)^2) / (2*sg^2))

#   Full KRR (n centers = n training points)
K_full <- k_rbf(x_train, x_train)
alpha  <- solve(K_full + lambda * diag(n)) %*% y_train
yhat_full <- k_rbf(x_test, x_train) %*% alpha

#   Sparse RBF (M << n centers from k-means)
M         <- 20
km        <- kmeans(x_train, centers = M, nstart = 30)
centers_s <- km$centers

K_sparse  <- k_rbf(x_train, centers_s)        # n x M
w_sparse  <- coef(lm(y_train ~ K_sparse - 1)) # M weights by OLS
yhat_sparse <- k_rbf(x_test, centers_s) %*% w_sparse

#   Residual comparison
cat("Full KRR   RMSE:", round(sqrt(mean((yhat_full  - sin(x_test))^2)), 4), "\n")
cat("Sparse RBF RMSE:", round(sqrt(mean((yhat_sparse - sin(x_test))^2)), 4), "\n")

par(mfrow = c(1, 2))
for (nm in c("Full KRR (n=150)", "Sparse RBF (M=20)")) {
  yhat <- if (grepl("Full", nm)) yhat_full else yhat_sparse
  plot(x_train, y_train, pch = 16, cex = 0.4, col = "gray70",
       main = nm, xlab = "x", ylab = "y", ylim = c(-1.6, 1.6))
  lines(x_test, sin(x_test), lwd = 2, lty = 2)
  lines(x_test, yhat,        lwd = 2, col = "steelblue")
  if (grepl("Sparse", nm))
    abline(v = centers_s, col = "coral", lty = 3, lwd = 0.8)
}
```

9 RBF Networks as Shallow Neural Networks

An RBF network is a two-layer neural network:

$$\boldsymbol{x} \;\longrightarrow\; \big\{\phi(\|\boldsymbol{x}-\boldsymbol{c}_j\|)\big\}_{j=1}^{M} \;\longrightarrow\; f(\boldsymbol{x}) = \sum_j w_j \phi(\|\boldsymbol{x}-\boldsymbol{c}_j\|).$$

The hidden layer is nonlinear with localized activations; the output layer is linear. Training approaches include: (i) fix centers (e.g., via k-means) and optimize weights w_j by least squares; or (ii) jointly optimize centers and weights by gradient descent.

R Example: Interpreting the RBF Hidden Layer as Feature Maps

```
set.seed(55)
n  <- 100
x  <- runif(n, -3, 3)
y  <- exp(-x^2) + 0.15 * rnorm(n)
```

```
M       <- 8
centers <- seq(-3, 3, length = M)
sigma   <- 0.8

# Hidden layer activations (feature map phi(x))
Phi <- outer(x, centers, function(xi, cj)
  exp(-(xi - cj)^2 / (2 * sigma^2)))   # n x M

# Linear output layer
w    <- coef(lm(y ~ Phi - 1))

# Visualise the hidden unit responses (basis functions)
x_grid <- seq(-3, 3, length = 300)
Phi_g  <- outer(x_grid, centers, function(xi, cj)
  exp(-(xi - cj)^2 / (2 * sigma^2)))
yhat_g <- Phi_g %*% w

par(mfrow = c(1, 2))

# Hidden unit profiles
matplot(x_grid, Phi_g, type = "l", lty = 1,
        main = "Hidden unit activations phi_j(x)",
        xlab = "x", ylab = expression(phi[j](x)))
abline(v = centers, lty = 3, col = "gray50")

# Final fit
plot(x, y, pch = 16, cex = 0.6, col = "gray70",
     main = "RBF network fit",
     xlab = "x", ylab = "y")
lines(x_grid, exp(-x_grid^2), lwd = 2, lty = 2)
lines(x_grid, yhat_g,         lwd = 2, col = "darkgreen")
legend("topright", legend = c("Truth", "RBF fit"),
       lty = c(2, 1), col = c("black", "darkgreen"), bty = "n")
```

10 Neural Networks and RKHS

10.1 Shallow Networks

For fixed hidden-layer parameters, the shallow network $f(\boldsymbol{x}) = \sum_{j=1}^{M} a_j \sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)$ is a linear model in the feature space $\{\sigma(\boldsymbol{w}_j^\top \boldsymbol{x} + b_j)\}$. Thus shallow networks with fixed parameters behave like kernel machines with a kernel implicitly defined by the activation function and weight distribution.

10.2 Deep Networks

Deep networks differ fundamentally: representations are hierarchical, feature spaces are fully data-adaptive, and optimization is nonconvex. Two important bridges to kernel theory nonetheless exist:

- The *Neural Tangent Kernel* (NTK) framework shows that infinitely wide networks trained by gradient descent converge to kernel regression with a specific kernel determined by the network architecture.
- As shown in Section 2, random infinite-width networks converge to Gaussian processes, linking deep nets to RKHS function spaces.

Remark 10.1. In the infinite-width limit, a neural network is equivalent to a kernel machine. Finite-width networks interpolate between the kernel regime and fully adaptive feature learning.

Why RKHS Theory Is Useful

Why kernels work. Kernels implicitly define feature maps via $K(\boldsymbol{x}, \boldsymbol{x}') = \langle \phi(\boldsymbol{x}), \phi(\boldsymbol{x}') \rangle$, so learning with kernels is equivalent to linear regression in a (possibly infinite-dimensional) feature space $f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x})$. RKHS theory formalizes the function space in which this regression takes place.

Why regularization is necessary. Without the penalty $\lambda \|f\|_{\mathcal{H}}^2$, perfect interpolation is generally possible in an RKHS, and the estimator is unstable. Regularization controls smoothness and ensures stable estimation.

Why solutions have finite representations. The Representer Theorem shows that despite optimizing over an infinite-dimensional space, the solution has the form $f(\boldsymbol{x}) = \sum_{i=1}^n \alpha_i K(\boldsymbol{x}_i, \boldsymbol{x})$.

How classical smoothing relates to modern machine learning. Smoothing splines solve

$$\min_f \sum_{i=1}^n (y_i - f(x_i))^2 + \lambda \int (f''(x))^2 \, dx,$$

which is an RKHS optimization problem. Kernel ridge regression and GP regression are also RKHS methods with different kernels. RKHS theory thus unifies classical statistical methods and modern machine learning as instances of regularized optimization in a function space.

R Example: The Neural Tangent Kernel of a Shallow ReLU Network

```
# The NTK for a width-m ReLU network approximated by Monte Carlo.
# As m -> infinity this converges to the arc-cosine kernel.

set.seed(42)
relu     <- function(z) pmax(0, z)
relu_d   <- function(z) as.numeric(z > 0)   # subgradient

ntk_empirical <- function(x1, x2, m = 5000) {
  W <- rnorm(m); b <- rnorm(m)
  h1 <- relu(W * x1 + b); h2 <- relu(W * x2 + b)
  I1 <- relu_d(W * x1 + b); I2 <- relu_d(W * x2 + b)
  # NTK = E[phi(x1)phi(x2)] + E[x1*x2 * I(h1>0)*I(h2>0)]
  mean(h1 * h2) + x1 * x2 * mean(I1 * I2)
}

arc_cosine_kernel <- function(x1, x2) {
  n1 <- abs(x1); n2 <- abs(x2)
```

```
  K12 <- x1 * x2
  theta <- acos(pmin(pmax(K12 / (n1 * n2), -1), 1))
  1 / (2 * pi) * n1 * n2 * (sin(theta) + (pi - theta) * cos(theta))
}

# Evaluate on a grid of pairs
grid <- seq(-2, 2, by = 0.5)
pairs <- expand.grid(x1 = grid, x2 = grid)
pairs$NTK_emp    <- mapply(ntk_empirical, pairs$x1, pairs$x2)
pairs$Arc_cosine <- mapply(arc_cosine_kernel, pairs$x1, pairs$x2)
pairs$error      <- abs(pairs$NTK_emp - pairs$Arc_cosine)

cat("Max |NTK_empirical - Arc-cosine|:",
    round(max(pairs$error, na.rm = TRUE), 4), "\n")

# A practical demonstration: KRR with NTK vs arc-cosine kernel
x_tr <- runif(40, -2, 2)
y_tr <- sin(pi * x_tr) + 0.2 * rnorm(40)
x_te <- seq(-2, 2, length = 200)

build_K <- function(x1, x2, kfun)
  outer(x1, x2, Vectorize(kfun))

lam <- 0.05
for (kname in c("NTK_emp", "Arc_cosine")) {
  kf <- if (kname == "NTK_emp") ntk_empirical else arc_cosine_kernel
  K_tr   <- build_K(x_tr, x_tr, kf)
  K_te   <- build_K(x_te, x_tr, kf)
  alpha  <- solve(K_tr + lam * diag(40)) %*% y_tr
  assign(paste0("yhat_", kname), K_te %*% alpha)
}

plot(x_tr, y_tr, pch = 16, cex = 0.6, col = "gray70",
     main = "KRR with NTK vs arc-cosine kernel",
     xlab = "x", ylab = "y")
lines(x_te, sin(pi * x_te),     lwd = 2, lty = 2)
lines(x_te, yhat_NTK_emp,       lwd = 2, col = "steelblue")
lines(x_te, yhat_Arc_cosine,    lwd = 2, col = "firebrick", lty = 3)
legend("topright",
       legend = c("Truth", "NTK (empirical)", "Arc-cosine"),
       lty = c(2, 1, 3),
       col = c("black", "steelblue", "firebrick"), bty = "n")
```

11 Unifying View

The methods discussed form a conceptual ladder:

$$\text{RKHS} \Rightarrow \text{Kernel methods} \Rightarrow \text{RBF networks} \Rightarrow \text{Neural networks}.$$

Each step relaxes a constraint of the previous one:

- **RKHS / kernel methods:** convex optimization in function space; implicit infinite-dimensional feature map; explicit regularization.
- **RBF networks:** finite kernel expansions with learned or fixed centers; bridges kernel methods and neural networks.
- **Shallow neural networks:** learned feature maps with nonconvex optimization; recover kernel machines in the infinite-width limit.
- **Deep neural networks:** hierarchical, fully data-adaptive representations.

R Example: Side-by-Side Comparison of All Methods

```
# Fit the same 1-D regression problem with every method
# discussed in these notes, and compare on a test grid.
set.seed(2024)
n       <- 100
x_tr    <- runif(n, 0, 2 * pi)
y_tr    <- sin(x_tr) + sin(2 * x_tr) + 0.3 * rnorm(n)
x_te    <- seq(0, 2 * pi, length = 400)
f_true  <- sin(x_te) + sin(2 * x_te)

sg <- 0.4; lam <- 0.01

k_rbf <- function(x1, x2)
  exp(-outer(x1, x2, function(a,b)(a-b)^2) / (2*sg^2))

# 1. Smoothing spline (RKHS via smooth.spline)
ss  <- smooth.spline(x_tr, y_tr, cv = TRUE)
yhat_ss <- predict(ss, x_te)$y

# 2. Natural spline basis regression
library(splines)
fit_ns   <- lm(y_tr ~ ns(x_tr, df = 10))
yhat_ns  <- predict(fit_ns,
                    newdata = data.frame(x_tr = x_te))

# 3. Kernel ridge regression (full KRR)
K_tr   <- k_rbf(x_tr, x_tr)
alpha  <- solve(K_tr + lam * diag(n)) %*% y_tr
yhat_krr <- k_rbf(x_te, x_tr) %*% alpha

# 4. Sparse RBF network (k-means centers)
km      <- kmeans(x_tr, centers = 15, nstart = 20)
ctr     <- km$centers
Phi_tr  <- k_rbf(x_tr, ctr)
w_rbf   <- coef(lm(y_tr ~ Phi_tr - 1))
yhat_rbf <- k_rbf(x_te, ctr) %*% w_rbf
```

```
# 5. Shallow neural network
library(nnet)
fit_nn  <- nnet(x_tr, y_tr, size = 20, linout = TRUE,
                maxit = 1000, decay = 1e-3, trace = FALSE)
yhat_nn <- predict(fit_nn, data.frame(x_tr = x_te))

# RMSE table
methods <- c("Smoothing spline", "Natural spline",
             "KRR (RKHS)", "Sparse RBF", "Neural net")
yhats   <- list(yhat_ss, yhat_ns, yhat_krr, yhat_rbf, yhat_nn)
rmse    <- sapply(yhats, function(yh) sqrt(mean((yh - f_true)^2)))
print(data.frame(Method = methods, RMSE = round(rmse, 4)))

# Plot
par(mfrow = c(2, 3))
cols <- c("dodgerblue","forestgreen","firebrick","darkorange","purple")
for (i in seq_along(methods)) {
  plot(x_tr, y_tr, pch = 16, cex = 0.4, col = "gray70",
       main = methods[i], xlab = "x", ylab = "y",
       ylim = c(-2.5, 2.5))
  lines(x_te, f_true,     lwd = 2, lty = 2)
  lines(x_te, yhats[[i]], lwd = 2, col = cols[i])
}
```

12 Takeaways

- RKHS theory provides the mathematical framework for learning in function spaces.
- The Representer Theorem shows why infinite-dimensional kernel optimization reduces to a finite linear system.
- RBF networks are finite-dimensional approximations to Gaussian RKHS regression.
- Neural networks generalize kernel machines by learning the feature representation.
- Modern theory connects deep learning back to RKHS via the Neural Tangent Kernel and Gaussian process limits.

Final message:

> RKHS theory is not an alternative to neural networks—it is their mathematical ancestor.

References

Youngmin Cho and Lawrence Saul. Kernel methods for deep learning. In *Advances in Neural Information Processing Systems*, 2009.

Jaehoon Lee, Yasaman Bahri, Roman Novak, Samuel Schoenholz, Jeffrey Pennington, and Jascha Sohl-Dickstein. Deep neural networks as gaussian processes. *International Conference on Learning Representations*, 2018.

Radford Neal. *Bayesian Learning for Neural Networks*. Springer, 1996.

Christopher K. I. Williams. Computation with infinite neural networks. *Neural Computation*, 10 (5):1203–1216, 1997.